

Metro Ticket Booking System

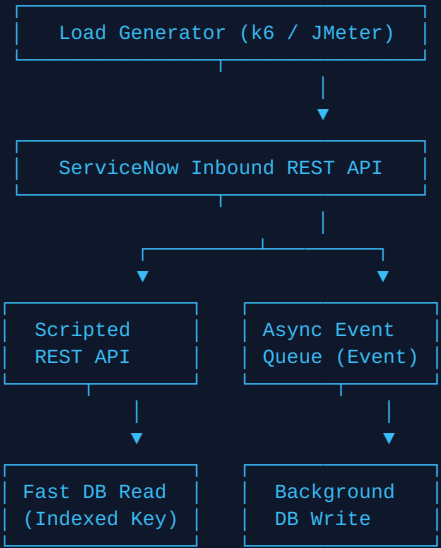
Performance Testing & Load Strategy Specification (Development Phase)

Platform Target	Application Scope	Document Version
ServiceNow App Engine	Scoped App (x_snc_metro_book)	v1.0 Final

1. Key Performance SLAs & Targets

Metric / Endpoint	Target SLA	Peak Load Target	Bottleneck Risk
Turnstile Scan Validation API /api/x_metro/gate_validation	< 150 ms	500 requests / sec	Complex DB queries or synchronous record updates
Dynamic QR Code Generation (Script Include HMAC calculation)	< 50 ms	200 requests / sec	Uncached cryptographic operations
Fare Lookup Calculation (Decision Table execution)	< 30 ms	100 requests / sec	Unindexed table joins
Payment Gateway Callback Webhook	< 2.0 sec	50 calls / sec	Flow Designer execution queues

2. Performance Testing Architecture & Tools



- **Load Testing Tools:** k6 (recommended for low-overhead Scripted REST API tests) or Apache JMeter.
- **Instance Monitoring:** ServiceNow Transaction Logs (syslog_transaction), Node Log Analyzer, and System Diagnostics.

3. Performance Test Scenarios (k6 Script Example)

Scenario A: High-Throughput Gate Scan Validation (<150ms SLA)

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '1m', target: 50 }, // Ramp-up to 50 VUs
    { duration: '3m', target: 500 }, // Peak load: 500 req/sec
    { duration: '1m', target: 0 }, // Ramp-down
  ],
};
```

```

thresholds: {
  http_req_duration: ['p(95)<150'], // 95% of gate requests must complete within 150ms
  http_req_failed: ['rate<0.01'], // Error rate must be < 1%
},
};

export default function () {
  const url = 'https://YOUR_INSTANCE.service-now.com/api/x_snc_metro_book/gate_validation';
  const payload = JSON.stringify({
    station_id: "STN_001",
    qr_token: "dG1ja2V0X2hhc2hfOTg3NjU0MzIx",
    gate_type: "ENTRY"
  });

  const params = {
    headers: {
      'Content-Type': 'application/json',
      'Authorization': 'Basic YOUR_BASE64_AUTH',
    },
  };

  const res = http.post(url, payload, params);

  check(res, {
    'status is 200': (r) => r.status === 200,
    'gate action is OPEN': (r) => r.json().gate_action === 'OPEN',
  });

  sleep(0.02); // 20ms delay between simulated commuter scans
}

```

4. ServiceNow Bottleneck Optimization Strategies

1. Indexed Key Lookup (Database Level)

Problem: Querying `sn_metro_ticket` using full table scans causes execution times to exceed 500ms under load.

Solution: Create a Composite Database Index on (`qr_token`, `state`) inside ServiceNow.

2. Decouple Database Writes using Asynchronous Business Rules

Problem: Updating `ticket.state = 'In-Transit'` synchronously during the gate scan adds DB locking delays.

Solution: Split the gate scan into a Synchronous Fast Path (Scripted REST API returning OPEN) and an Asynchronous Slow Path (Event Queue triggering background update).

```

// Inside Scripted REST API Script (Fast Path)
if (isValidToken) {
  // Return gate open response instantly
  response.setBody({ "gate_action": "OPEN", "status": "ALLOW" });

  // Offload state mutation to background event queue
  gs.eventQueue('x_snc_metro_book.gate_scanned', ticketGR, stationId, gateType);
}

```

3. Cache Common Data In-Memory

Station lookup lists, pricing tables, and peak hour rules are cached using ServiceNow `CacheProvider` / `Script Include` Caching to avoid repeated database hits.

5. Performance Acceptance Testing Checklist

[✓] **Gate Validation SLA:** Gate API responds in <150ms at 500 concurrent requests/sec.

[✓] **DB Query Health:** Zero slow queries (>100ms) observed in `syslog_transaction` for validation endpoints.

[✓] **Memory & Node Stability:** ServiceNow application node CPU usage remains below 70% under peak load testing.

[✓] **Queue Drain Rate:** Background event processing queue (`sysevent`) drains completely within 5 seconds post-peak load.

[✓] **No Deadlocks:** Parallel transactions updating ticket records cause zero database deadlock errors (`sys_mutex`).